



## Programmation

UE 4 - Informatique

2025-26

### Travaux dirigés n°2

#### Fonctions avec d'autres fonctions

#### Introduction

En Python, une fonction peut en appeler une autre, ou même prendre une fonction comme argument et/ou retourner une fonction. C'est ce qui permet de composer et de réutiliser du code.

Voici les principaux cas :

##### 1. Une fonction qui appelle une autre

C'est le cas le plus simple : une fonction utilise une autre pour effectuer une partie de son travail.

```
def carre(x):  
    return x * x  
  
def somme_carres(a, b):  
    return carre(a) + carre(b)  
  
print(somme_carres(2, 3)) # Résultat : 13
```

Ici, `somme_carres()` utilise la fonction `carre()`.

##### 2. Passer une fonction comme argument

En Python, les fonctions sont des objets comme les autres : on peut les passer en paramètre.

```
def appliquer_operation(operation, x, y):  
    return operation(x, y)  
  
def addition(a, b):  
    return a + b  
  
def multiplication(a, b):  
    return a * b  
  
print(appliquer_operation(addition, 4, 5)) # Résultat : 9  
print(appliquer_operation(multiplication, 4, 5)) # Résultat : 20
```

Ici, `appliquer_operation()` prend une fonction en argument.



### 3. Retourner une fonction (fonction imbriquée)

Une fonction peut aussi créer et renvoyer une autre fonction.

```
def creer_multiplicateur(facteur):  
    def multiplier(x):  
        return x * facteur  
    return multiplier  
  
double = creer_multiplicateur(2)  
triple = creer_multiplicateur(3)  
  
print(double(5)) # Résultat : 10  
print(triple(5)) # Résultat : 15
```

Ceci est la base des **fonctions de haut niveau** (higher-order functions) et des **closures**

#### 1. Fonction de conversion appelant une autre fonction

Créer une fonction **energie\_masse()** qui calcule l'énergie équivalente d'une masse **m** selon  $E = mc^2$ . Puis enchaîner avec une fonction **energie\_en\_kwh()** qui renvoie le résultat en kilowatt-heure.

Conversion joule – kilowatt-heure :  $1 \text{ J} = 2,7778 \cdot 10^{-7} \text{ kWh}$

#### 2. Fonction avec d'autres fonctions en argument

Objectif : Ecrire une fonction **apply\_func()** qui prend en argument une fonction **func()** et un nombre **x**, et qui retourne le résultat de **func(x)**.

Q1. Créer une fonction **apply\_func()** qui prend en argument une fonction **func()** et un nombre **x**, et qui retourne le résultat de l'application de **func()** à **x**.

Q2. Tester la fonction **apply\_func()** avec les fonctions suivantes :

- **square()** – une fonction qui renvoie le carré de **x**
- **cube()** – une fonction qui renvoie le cube de **x**

#### 3. Fonctions logarithmes imbriqués

Ecrire une fonction **formula()** qui calcule la formulation mathématique suivante :

$$f(x) = \ln \left( \sqrt{\ln(e^{x^2}) + 1} \right),$$

en utilisant uniquement des fonctions imbriquées importées du module **math**, soit :

- **log()** – la fonction qui sert à calculer le **logarithme** d'un nombre.
- **sqrt()** – la fonction qui calcule la **racine carrée** d'un nombre.
- **exp()** – la fonction mathématique qui calcule l'**exponentielle** d'un nombre.



#### 4. Nombres complexes et exponentielle (niveau un peu plus avancé)

Ecrire une fonction `h_complex()` qui prend en argument les parties réelle, **a**, et imaginaire, **b**, d'un nombre complexe :

$$z = a + bj,$$

et qui renvoie :

$$|e^z| + \arg(e^z), \text{ où } \arg \text{ est l'argument du nombre complexe.}$$

On utilisera les fonctions imbriquées importées du module `cmath`.